

INFORME DE AUDITORÍA Y PLAN MAESTRO DE MIGRACIÓN

AxonBIM → AxonBIM Web

Versión: 1.0

Estado: Documento de planificación previo al desarrollo

Propósito: establecer la base técnica, arquitectónica y metodológica del nuevo AxonBIM Web antes de crear `AGENTS.md` y `.cursor/rules/`.

1. Resumen ejecutivo

El proyecto anterior de AxonBIM constituye una base técnica valiosa, pero no debe ser trasladado literalmente al nuevo producto.

La auditoría del repositorio confirma que el desarrollo anterior posee componentes reales y reutilizables relacionados con:

- modelo paramétrico;
- geometría analítica;
- generación de mallas;
- muros y huecos;
- losas;
- snapping;
- trazado encadenado;
- historial;
- pruebas geométricas;
- JSON-RPC;
- IFC;
- identidad de elementos;
- documentación arquitectónica;
- reglas de trabajo para agentes.

La arquitectura anterior también demuestra que varias decisiones fundamentales fueron correctas:

Godot muestra; Python decide.

Sin embargo, el proyecto acumuló demasiadas responsabilidades simultáneamente: frontend, backend, RPC, IFC, normativa, persistencia, distribución, topología, GPU, documentación y herramientas de modelado.

El problema principal no fue una mala base técnica.

Fue la **acumulación de complejidad y alcance antes de consolidar suficientemente la experiencia de usuario.**

El nuevo proyecto no debe ser una migración de código Python/Godot hacia TypeScript/React/Three.js.

Debe ser una **reconstrucción controlada basada en comportamientos validados**.

La nueva regla arquitectónica será:

El documento paramétrico es la fuente de verdad. Los comandos lo modifican. Las vistas representan sus resultados.

Y la nueva regla metodológica será:

Cada fase debe producir una mejora visible, manipulable y comprensible dentro de la aplicación.

2. Resultado de la auditoría del proyecto anterior

2.1. Estado general

La inspección directa del repositorio confirma que el proyecto anterior es sustancial y contiene una arquitectura real, no un simple prototipo.

Entre los elementos verificados se encuentran:

- código Python de producción;
- código GDScript;
- tests;
- documentación arquitectónica;
- ADR;
- reglas `.cursor`;
- integración IFC;
- sistema RPC;
- historial;
- geometría analítica;
- herramientas de modelado;
- frontend Godot.

Por tanto, las afirmaciones de la auditoría anterior relativas a la estructura general del proyecto están suficientemente respaldadas.

3. Componentes heredados confirmados

3.1. Modelo geométrico

El repositorio contiene estructuras como:

- WallSpec ;
- WallOpeningSpec ;
- SlabSpec ;
- WorkspaceXYHalfExtents .

También existen generadores relacionados con:

- wall_box_mesh ;
- wall_box_mesh_with_openings ;
- slab_prism_mesh ;
- wall_mesh_for_spec .

Esto confirma que el proyecto anterior ya poseía una separación significativa entre:

Especificación paramétrica
↓
Geometría analítica
↓
Representación

Esta separación es uno de los activos principales que deben conservarse conceptualmente.

4. Geometría analítica

La auditoría confirma que el proyecto anterior utiliza un pipeline geométrico analítico propio para elementos arquitectónicos simples.

Esto resulta especialmente adecuado para:

- muros rectos;
- prismas;
- losas simples;
- huecos rectangulares;
- extrusiones;
- proyecciones.

No se debe interpretar esta arquitectura como un kernel B-Rep completo.

La geometría anterior debe considerarse:

una colección de reglas geométricas especializadas y probadas, no un motor CAD general.

5. OpenCascade

La ausencia de OpenCascade como dependencia central del pipeline actual refuerza la conclusión de que abandonar un kernel CAD pesado para la interacción cotidiana fue razonable.

Por tanto:

Decisión para AxonBIM Web

OpenCascade no formará parte del MVP.

Su posible incorporación futura requerirá:

1. un problema concreto;
2. una necesidad que la geometría analítica no pueda resolver razonablemente;
3. un análisis de coste;
4. un ADR;
5. autorización explícita.

No se incorporará por anticipación.

6. Arquitectura frontend/backend heredada

El proyecto anterior establece explícitamente una separación:

Godot

↓

representación / interacción

y:

Python

↓

geometría / modelo / IFC / persistencia / decisiones

La auditoría confirma esta separación.

Fue una decisión correcta.

El problema fue que la separación evolucionó hacia una infraestructura compleja que incluía:

- dos procesos;
- JSON-RPC;
- framing;
- reconexión;
- sincronización;
- estado distribuido;
- lógica de frontend;
- lógica de backend.

Para una aplicación web local-first, esa frontera ya no debe existir obligatoriamente.

7. JSON-RPC

El JSON-RPC anterior es real y está tipado/documentado.

También existe el flujo de operaciones geométricas y respuestas que incluyen información de malla y topología.

Sin embargo:

Decisión

JSON-RPC no será trasladado como arquitectura interna del nuevo producto.

En AxonBIM Web:

```
UI
↓
Tool
↓
Command
↓
AxonDocument
↓
Geometry
↓
Representation
```

Las operaciones se realizarán dentro del mismo entorno TypeScript salvo que posteriormente exista una necesidad real de Worker o servicio externo.

8. Topología: corrección importante

La auditoría directa encontró una diferencia entre documentación e implementación.

La documentación de topología describe mecanismos como:

```
compute_topo_id  
propagate_topology
```

pero `compute_topo_id` sí está implementado mientras que `propagate_topology` no fue encontrado en la implementación actual.

Esto demuestra que:

La documentación del proyecto anterior no puede considerarse automáticamente equivalente al estado real del código.

Por tanto, en la migración:

No se debe portar una característica solamente porque aparezca documentada.

Cada elemento deberá clasificarse como:

- implementado y probado;
- implementado parcialmente;
- documentado pero no implementado;
- experimental;
- obsoleto;
- descartado.

9. Topología: decisión para AxonBIM Web

No se intentará reproducir inicialmente el sistema de nombrado topológico B-Rep general.

La identidad estable de elementos sí es necesaria.

Pero debe distinguirse:

```
Identidad del elemento
```

de:

```
Identidad universal de cada cara/arista/vértice B-Rep
```

El MVP trabajará con identidad estable de objetos arquitectónicos.

La topología avanzada se aplazará hasta existir una funcionalidad que realmente la necesite.

10. Historial

El proyecto anterior posee historial basado en SQLite y operaciones reversibles.

La idea funcional debe conservarse:

- undo;
- redo;
- operaciones reversibles;
- identidad estable;
- recuperación coherente.

Pero no se copiará directamente SQLite.

La nueva arquitectura utilizará comandos.

Conceptualmente:

```
Command
├─ execute()
└─ undo()
```

El historial será responsabilidad del sistema de comandos.

La persistencia del historial podrá evolucionar posteriormente.

11. Pruebas heredadas

El proyecto anterior contiene una base significativa de pruebas relacionadas con:

- geometría;
- RPC;
- IFC;
- historial;
- integración;
- estrés.

La estrategia será:

portar comportamiento y casos de prueba, no archivos completos.

Además, las pruebas geométricas se utilizarán como referencia normativa para verificar que la nueva implementación conserva los comportamientos relevantes.

12. Qué significa "portar" realmente

No significa:

```
Python
↓
traducción automática
↓
TypeScript
```

Significa:

```
Código anterior
↓
Comportamiento observado
↓
Caso de prueba
↓
Invariante
↓
Especificación
↓
Nueva implementación
```

Esta diferencia es fundamental.

13. Equivalencia geométrica

No se exigirá que la nueva implementación produzca exactamente:

- el mismo orden de vértices;
- los mismos índices;
- la misma triangulación;
- el mismo número de triángulos.

Dos implementaciones pueden producir geometría equivalente con representaciones internas diferentes.

La validación deberá centrarse en invariantes:

- dimensiones;

- longitud;
- altura;
- espesor;
- posición;
- orientación;
- bounds;
- área cuando corresponda;
- volumen cuando corresponda;
- relaciones geométricas;
- comportamiento de huecos.

Por tanto:

Se validará equivalencia geométrica, no identidad accidental de implementación.

14. Principios que se conservan del proyecto anterior

Se conservarán conceptualmente:

1. Modelo autoritativo.
 2. Geometría analítica.
 3. Separación entre modelo y representación.
 4. Identidad estable.
 5. Operaciones reversibles.
 6. Pruebas geométricas.
 7. Manejo explícito de errores.
 8. Trazado sobre un plano matemático.
 9. Generación de representación a partir del modelo.
 10. Documentación de decisiones.
-

15. Elementos que no se portarán como arquitectura

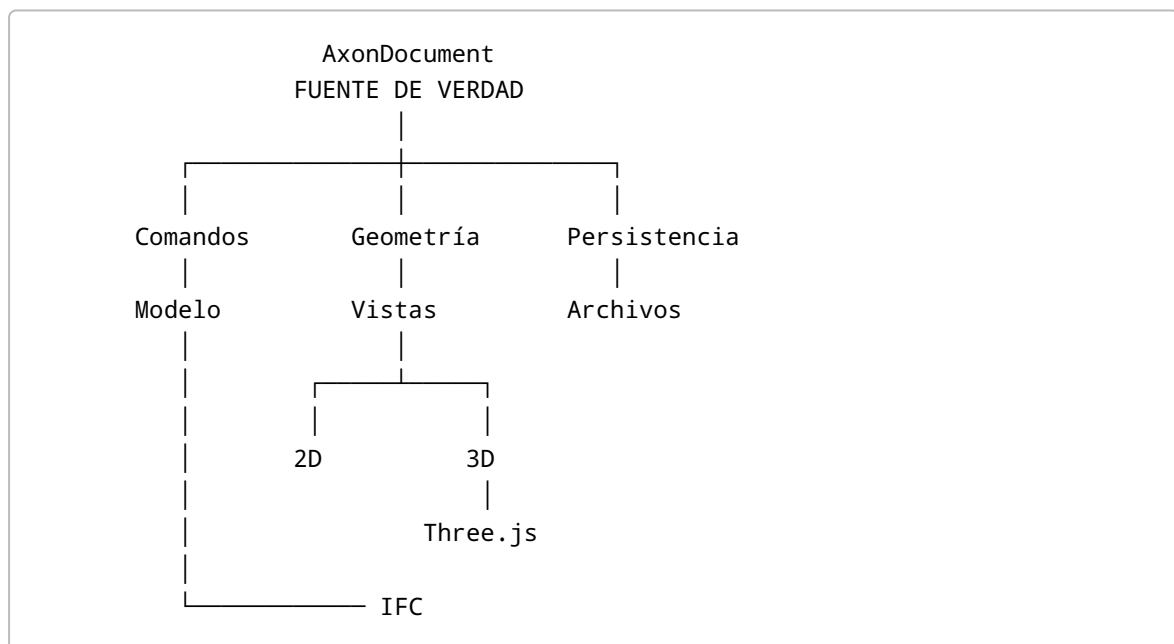
No se portarán directamente:

- Godot;
- `main_scene.gd`;
- JSON-RPC;
- TCP;
- framing LSP;
- reconexión;
- proceso Python obligatorio;
- servidor local obligatorio;
- worker Godot headless;

- configuración específica de GPU;
- scripts específicos de Flatpak;
- mutación de IFC durante cada gesto;
- infraestructura de distribución anterior;
- arquitectura B-Rep general.

16. Principio arquitectónico central de AxonBIM Web

La nueva arquitectura será:



Regla fundamental:

El documento no conoce React, Three.js, IndexedDB, IFC, OpenCascade ni APIs del navegador.

17. Documento paramétrico

El objeto central será `AxonDocument`.

Conceptualmente:

```

AxonDocument
├─ metadata
├─ units
├─ storeys

```

└─ elements
└─ version

Los elementos iniciales incluirán principalmente:

Wall

y posteriormente:

WallOpening
Door
Window
Slab
Storey

18. Sistema de coordenadas

Se establecerá formalmente:

Unidad interna: metros
Plano de planta: XY
Eje vertical: Z
Ángulos internos: radianes

También deberán definirse antes de implementar herramientas:

- orientación positiva;
- sistema de coordenadas derecho;
- origen;
- transformación pantalla → mundo;
- transformación mundo → pantalla;
- plano de trabajo;
- tolerancias;
- precisión numérica.

19. Tolerancias geométricas

Este punto se incorpora como requisito formal.

Antes de desarrollar snapping deberán definirse:

- tolerancia de igualdad;

- tolerancia de snapping;
- tolerancia de coincidencia de extremos;
- distancia mínima de muro;
- tolerancia angular;
- tolerancia de cierre;
- precisión de serialización.

No se permitirá que cada herramienta invente sus propias tolerancias.

Existirá una política geométrica común.

20. Herramientas y comandos

Las herramientas serán responsables de interacción.

Los comandos serán responsables de mutación confirmada.

Ejemplo:

```
WallTool
  ↓
CreateWallCommand
  ↓
AxonDocument
```

La herramienta nunca modificará directamente el documento.

21. Previsualización

La previsualización durante el trazado no forma parte del documento hasta que el usuario confirma la operación.

Por tanto:

```
cursor
  ↓
preview
```

no genera historial.

Solo:

```
confirmación
↓
command
↓
document
```

entra en el modelo.

22. Three.js

Three.js será un adaptador de representación.

La dirección será:

```
Wall
↓
WallGeometryBuilder
↓
BufferGeometry
↓
Three.js Mesh
```

Nunca:

```
Three.js Mesh
↓
fuente de verdad del Wall
```

Three.js puede tener estado visual:

- cámara;
- selección;
- hover;
- overlays;
- materiales;
- escena;

pero no será propietario del estado arquitectónico.

23. React

React será responsable principalmente de:

- interfaz;
- ribbon;
- propiedades;
- navegador;
- diálogos;
- catálogo;
- mensajes;
- estado visual.

React no modificará directamente las entidades del documento.

La operación será:

```
React
↓
Tool / acción
↓
Command
↓
Document
```

24. MVP estricto

Esta es la definición oficial del MVP.

Incluido

Proyecto

- nuevo proyecto;
- proyecto de demostración;
- formato propio versionado.

Muros

- crear muro;
- P1/P2;
- muros encadenados;
- snapping ortogonal;
- snapping a extremos;
- cierre básico;
- previsualización;

- cancelación con Escape.

Familias

Tres configuraciones iniciales:

- muro ligero — 100 mm;
- bloque — 150 mm;
- bloque — 200 mm.

Podrá existir espesor personalizado desde propiedades.

No habrá editor general de familias.

Edición

- seleccionar;
- resaltar;
- modificar altura;
- modificar espesor;
- cambiar familia;
- eliminar;
- undo;
- redo.

Vistas

- planta;
- perspectiva;
- ajustar modelo a vista.

Persistencia

- exportar JSON/ `.axon` ;
- importar JSON/ `.axon` ;
- recuperación temporal limitada;
- formato versionado.

Demo

Una vivienda sencilla compuesta exclusivamente por muros.

25. Fuera del MVP estricto

Quedan explícitamente fuera:

- puertas;
- ventanas;
- losas;
- múltiples niveles editables;

- medición;
- PWA;
- OPFS;
- IndexedDB como almacenamiento principal;
- IFC;
- DXF;
- cotas;
- documentación técnica;
- secciones;
- elevaciones documentales;
- OpenCascade;
- colaboración;
- multiusuario;
- renderizado fotorrealista.

Esta exclusión es obligatoria.

26. Criterio de éxito del MVP

El MVP estará terminado cuando puedas:

1. abrir AxonBIM Web;
2. abrir un proyecto vacío;
3. abrir la vivienda demo;
4. elegir una familia de muro;
5. dibujar varios muros encadenados;
6. cerrar un espacio;
7. seleccionar un muro;
8. cambiar altura;
9. cambiar espesor;
10. cambiar familia;
11. eliminar;
12. deshacer;
13. rehacer;
14. cambiar entre planta y perspectiva;
15. ajustar la vista;
16. exportar;
17. cerrar la aplicación;
18. importar nuevamente;
19. comprobar que el modelo conserva correctamente su información.

Todo esto deberá poder realizarse **sin leer código**.

27. El proyecto demo como requisito de producto

Desde las primeras versiones deberá existir una pequeña vivienda demostrativa.

Su función será:

- mostrar el avance;
- facilitar pruebas;
- validar selección;
- probar propiedades;
- detectar regresiones;
- evitar una aplicación vacía;
- permitir que el usuario evalúe inmediatamente el producto.

El demo será también un fixture funcional.

28. Estado de la maqueta

Durante prototipo y MVP existirá un pequeño panel:

ESTADO DE LA MAQUETA

- ✓ Crear muro
- ✓ Seleccionar
- ✓ Propiedades
- Muros encadenados
- Guardado
- ...

El panel no será una lista escrita arbitrariamente dentro de un componente.

Consumirá un manifest/versionado.

Después del MVP será sustituido por:

- changelog;
 - modal de novedades;
 - versión;
 - limitaciones conocidas.
-

29. Estrategia de persistencia

La persistencia será incremental.

Nivel 1 — Prototipo

Estado en memoria.

Proyecto demo incluido como JSON.

No se promete persistencia automática.

Nivel 2 — MVP

Exportación/importación JSON o `.axon`.

`localStorage` únicamente para:

- preferencias;
- último proyecto;
- estado temporal pequeño;
- configuración visual.

No será la base principal del modelo.

Nivel 3 — Persistencia local

Introducción de:

```
interface ProjectRepository {  
  save(document: AxonDocument): Promise<void>;  
  load(projectId: string): Promise<AxonDocument | null>;  
  list(): Promise<ProjectSummary[]>;  
  delete(projectId: string): Promise<void>;  
}
```

Primera implementación prevista:

IndexedDB.

Nivel 4 — OPFS

Solo cuando exista una necesidad real:

- assets grandes;
- texturas;
- modelos importados;
- backups;
- exportaciones grandes;
- contenedores complejos.

30. Migración: clasificación oficial

Cada componente heredado deberá tener una ficha:

Archivo anterior:
Responsabilidad:
Estado real:
Comportamiento útil:
Problema detectado:
Casos de prueba:
Invariantes:
Destino web:
Decisión:

Las decisiones serán:

CONSERVAR COMO CONCEPTO

La idea sigue siendo válida.

PORTAR PRUEBAS

El comportamiento debe mantenerse.

REESCRIBIR

La función es necesaria pero la implementación no.

MANTENER TEMPORALMENTE

Por ejemplo, determinadas herramientas IFC/DXF en Python.

APLAZAR

Es válida pero no necesaria todavía.

DESCARTAR

La complejidad no justifica conservarla.

VERIFICAR PRIMERO

La documentación o código no permiten determinar todavía su comportamiento con suficiente certeza.

31. Matriz inicial de migración

Componente anterior	Estado	Decisión
WallSpec	confirmado	Reescribir concepto
WallOpeningSpec	confirmado	Aplazar después del MVP

Componente anterior	Estado	Decisión
SlabSpec	confirmado	Aplazar
Geometría analítica	confirmada	Reimplementar
wall_box_mesh	confirmado	Portar comportamiento
wall_box_mesh_with_openings	confirmado	Aplazar
slab_prism_mesh	confirmado	Aplazar
Snapping	confirmado	Reimplementar
Trazado encadenado	confirmado	Reimplementar
Push/Pull	confirmado	Aplazar
JSON-RPC	confirmado	Descartar como arquitectura
SQLite history	confirmado	Rediseñar
topo_id	confirmado	Rediseñar
B-Rep topológico general	parcial/ documental	Aplazar
IFC	confirmado	Adaptador futuro
DXF	existente	Mantener temporalmente si se necesita
main_scene.gd	confirmado	No portar
Godot	confirmado	Sustituir
Python backend	confirmado	No portar estructuralmente
reglas .cursor antiguas	confirmadas	Analizar, no copiar

32. Primera fase de migración: inventario

Antes de escribir geometría nueva se creará:

```
docs/migration/legacy-inventory.md
```

Contendrá:

- módulos;
- funciones;
- responsabilidades;
- pruebas;
- dependencias;
- estado;

- decisión.

No se copiará todo el repositorio anterior al nuevo proyecto.

La carpeta de compatibilidad solo conservará:

- fixtures;
 - resultados esperados;
 - casos de prueba;
 - referencias puntuales;
 - explicaciones de comportamiento.
-

33. Fases de desarrollo

ETAPA 0 — Fundación visible

Objetivo

Crear un proyecto nuevo que pueda abrirse y evaluarse desde el primer día.

Entrega visible

- aplicación web;
- pantalla inicial;
- nuevo proyecto;
- abrir proyecto;
- abrir demo;
- layout básico;
- visor;
- propiedades;
- panel de estado.

Trabajo técnico

- TypeScript;
- pnpm;
- Vite;
- React;
- Three.js;
- Vitest;
- Playwright;
- documentación;
- sistema de coordenadas;
- unidades;
- ADR iniciales.

Criterio de salida

La aplicación abre y puede ser evaluada visualmente.

ETAPA 1 — Primer muro funcional

Objetivo

Conectar modelo, geometría, visor y UI mediante un corte vertical.

Flujo

```
Crear muro
↓
P1
↓
P2
↓
Wall
↓
Geometría
↓
Planta
↓
3D
↓
Selección
↓
Propiedades
```

Funciones

- crear;
- visualizar;
- seleccionar;
- cambiar altura;
- cambiar espesor;
- eliminar;
- undo;
- redo.

Criterio

El flujo completo funciona desde la interfaz.

ETAPA 2 — MVP estricto

Objetivo

Convertir el primer muro en una pequeña herramienta arquitectónica utilizable.

Añadir

- muros encadenados;
- snapping;
- cierre;
- tres familias;
- propiedades;
- demo;
- JSON;
- importación;
- exportación;
- pruebas visuales;
- pruebas geométricas.

Criterio

Debe poder dibujarse una pequeña vivienda utilizando solamente muros.

Después de esta etapa:

PAUSA OBLIGATORIA DE VALIDACIÓN DEL PRODUCTO

No se continuará automáticamente.

El usuario evaluará:

- sensación de dibujo;
- navegación;
- claridad;
- velocidad;
- aspecto;
- comportamiento;
- utilidad.

ETAPA 3 — Puertas y ventanas

Solo después de aprobar formalmente el MVP.

Funciones

- familias básicas;

- relación host-elemento;
 - previsualización;
 - colocación;
 - hueco;
 - dimensiones;
 - movimiento sobre muro;
 - undo/redo.
-

ETAPA 4 — Niveles y losas

Funciones

- crear niveles;
 - elevaciones;
 - nivel activo;
 - losa;
 - contorno;
 - espesor;
 - visibilidad;
 - dos niveles básicos.
-

ETAPA 5 — Herramientas complementarias

- mover;
 - medir;
 - duplicar;
 - navegador;
 - selección avanzada;
 - frente;
 - lateral;
 - ajustar vista.
-

ETAPA 6 — Persistencia avanzada y PWA

Solo después de que el modelado básico sea estable.

Funciones

- IndexedDB;
- `ProjectRepository`;
- recuperación;
- migraciones;
- PWA;
- offline;
- OPFS cuando exista justificación.

ETAPA 7 — IFC

IFC será un adaptador.

```
AxonDocument
  ↓
IFC Adapter
  ↓
IFC
```

Nunca:

```
Gesto
  ↓
IFC
  ↓
estado principal
```

La aplicación seguirá funcionando aunque el adaptador IFC falle.

ETAPA 8 — Documentación técnica

Posteriormente:

- plantas;
- rangos de vista;
- secciones;
- elevaciones;
- simbología;
- cotas;
- etiquetas;
- SVG;
- DXF;
- PDF;
- normativa MIVED progresiva.

La vista de modelado y la vista documental serán sistemas diferentes derivados del mismo documento.

34. Tecnología propuesta

Base

- TypeScript;
- pnpm;
- pnpm workspaces;
- Vite;
- React;
- Three.js;
- Vitest;
- Playwright;
- Zod;
- ESLint;
- Prettier.

Tecnologías futuras

- IndexedDB;
- OPFS;
- PWA;
- Web Workers.

Estas últimas no deben generar infraestructura antes de que la fase correspondiente las necesite.

35. Estructura inicial

La estructura inicial será deliberadamente pequeña:

```
axonbim-web/  
├── apps/  
│   └── web/  
├── packages/  
│   ├── model/  
│   ├── commands/  
│   ├── geometry/  
│   ├── tools/  
│   ├── viewer/  
│   ├── families/  
│   ├── persistence/  
│   └── shared/  
├── samples/  
│   └── demo-house/  
└── docs/
```

```
|   |─ product/
|   |─ architecture/
|   |─ decisions/
|   |─ validation/
|   └─ migration/
|
|─ .cursor/
|─ AGENTS.md
|─ CHANGELOG.md
└─ README.md
```

No se crearán inicialmente:

```
geometry-2d/
geometry-3d/
workers/
storage/
drawing-2d/
compatibility/
```

como paquetes independientes salvo que el tamaño y las dependencias lo justifiquen.

36. Responsabilidades

`packages/model`

Fuente de verdad:

- `AxonDocument` ;
- `Wall` ;
- `Storey` ;
- IDs;
- validación;
- serialización;
- migraciones.

No puede importar:

- React;
 - Three.js;
 - navegador;
 - IndexedDB.
-

packages/commands

Mutaciones:

- create;
 - update;
 - delete;
 - undo;
 - redo.
-

packages/geometry

Inicialmente:

- vectores;
- intersecciones;
- snapping;
- contornos;
- geometría de muro;
- bounds.

Se dividirá posteriormente si el tamaño lo justifica.

packages/tools

Máquinas de estados:

- WallTool;
- SelectionTool;
- ToolController.

No modifica directamente el documento.

packages/viewer

Adaptador Three.js:

- escena;
 - cámaras;
 - picking;
 - rejilla;
 - overlays;
 - mesh cache.
-

packages/families

Catálogo inicial de configuraciones.

No será todavía un editor general de familias.

packages/persistence

- JSON;
 - `.axon` ;
 - ProjectRepository;
 - recuperación;
 - IndexedDB posteriormente.
-

37. Desarrollo mediante cortes verticales

Cada funcionalidad debe atravesar solamente las capas necesarias.

Ejemplo:

```
Interfaz
↓
Herramienta
↓
Comando
↓
Modelo
↓
Geometría
↓
Representación
↓
Persistencia
↓
Prueba
```

Esto evita construir durante semanas componentes internos que todavía no puedan comprobarse.

38. Validación en tres niveles

Cada función tendrá tres estados:

1. Validación técnica

- tests;
- typecheck;
- lint;
- contratos;
- invariantes.

2. Validación funcional

Puede ejecutarse desde la aplicación.

3. Validación UX

El usuario puede evaluar:

- naturalidad;
- claridad;
- respuesta;
- apariencia;
- utilidad.

Una función no se considerará completamente terminada si solo pasa el nivel técnico.

39. Pruebas geométricas

Las pruebas serán principalmente deterministas.

Se verificarán:

- longitud;
- altura;
- espesor;
- bounds;
- orientación;
- vértices relevantes;
- contornos;
- relaciones;
- tolerancias.

No se utilizará el número exacto de triángulos como regla universal de equivalencia.

40. Pruebas de integración

Ejemplos:

Wall creado
↓
aparece en representación

Wall modificado
↓
solo su representación cambia

Wall eliminado
↓
representación eliminada

Cambio de cámara
↓
documento intacto

41. Pruebas visuales

Las capturas serán selectivas.

No se utilizarán snapshots como única prueba del sistema 3D porque pueden variar por:

- GPU;
- navegador;
- antialiasing;
- sistema operativo;
- densidad de píxeles.

Se utilizarán:

- screenshots controlados;
- proyecto demo;
- pruebas manuales;
- video corto cuando el movimiento sea importante.

42. Rendimiento Three.js

Desde el principio se respetará:

actualizar solamente lo afectado.

Ejemplo:

```
Cambio muro A
↓
recalcular muro A
↓
actualizar muro A
```

Nunca:

```
Cambio muro A
↓
borrar toda la escena
↓
regenerar todo
```

Se aplicarán:

- caché;
- geometrías compartidas;
- materiales compartidos;
- `dispose`;
- actualización selectiva.

La optimización para miles de elementos será posterior.

43. Política de agentes

Los agentes no serán responsables de decidir el producto.

Serán ejecutores controlados.

Cada tarea deberá declarar:

Objetivo visible

Qué podrá hacer el usuario.

Alcance

Qué entra y qué no entra.

Archivos afectados

Qué se modificará.

Criterios de aceptación

Cómo se comprobará.

Implementación

Solo lo acordado.

Verificación

Tests + demostración.

Informe

Qué cambió y qué no.

44. Regla de no expansión silenciosa

Un agente no podrá convertir una tarea pequeña en una oportunidad para:

- limpiar código;
- modernizar arquitectura;
- reorganizar carpetas;
- cambiar dependencias;
- refactorizar módulos;
- modificar componentes no relacionados.

Si encuentra una mejora:

1. termina la tarea actual;
 2. documenta la observación;
 3. propone una tarea separada;
 4. espera autorización.
-

45. Refactorizaciones

Ninguna refactorización de riesgo podrá ejecutarse por iniciativa del agente.

Requiere:

Problema:
Resultado esperado:
Archivos:
Contratos afectados:
Comportamientos que permanecen:
Riesgos:

Pruebas :
Tamaño :
Reversión :

Y autorización explícita:

Sí, autoriza este plan.

"Continúa con el proyecto" no será autorización para futuras refactorizaciones.

46. Presupuesto de cambio

Como orientación:

- 2-5 archivos;
- 1-3 grupos de pruebas;
- documentación solo si cambia contrato o comportamiento.

Estos valores no serán límites rígidos.

Un cambio mayor deberá justificarse antes de ejecutarse.

47. Eficiencia de tokens

La eficiencia no significa eliminar comprensión necesaria.

Regla:

El agente debe minimizar exploración innecesaria, pero nunca omitir contexto necesario para realizar correctamente la tarea.

No deberá:

- releer todo el repositorio sin necesidad;
- generar documentación innecesaria;
- modificar archivos no relacionados;
- analizar componentes irrelevantes.

Pero deberá leer todos los archivos necesarios para comprender el contrato que va a modificar.

48. Política de commits

Preferiblemente:

un cambio lógico por commit.

No se agruparán automáticamente:

```
función nueva
+
refactor
+
limpieza
+
actualización de arquitectura
+
cambio de dependencias
```

en un solo cambio.

49. Evidencia obligatoria

Cada entrega visible deberá indicar:

Qué cambió

Explicación no técnica.

Cómo probarlo

Pasos concretos.

Qué debería observarse

Resultado esperado.

Qué todavía no hace

Limitaciones.

Evidencia técnica

Tests y validaciones.

Evidencia visual

Una o más de:

- captura;
- demo;
- screenshot Playwright;

- video;
 - comparación antes/después.
-

50. El agente debe detenerse

Una vez cumplido:

- alcance;
- aceptación;
- pruebas;
- demostración;

el agente debe detenerse.

No debe continuar:

"mejorando"

sin autorización.

51. Clasificación de decisiones

Cada decisión del proyecto deberá pertenecer a una categoría.

HEREDADO

Comportamiento confirmado del proyecto anterior.

APRENDIZAJE

Conclusión obtenida de la experiencia anterior.

DECISIÓN APROBADA

Decisión tomada para AxonBIM Web.

PROPUESTA

Idea futura aún revisable.

Esto evitará que un agente interprete una propuesta futura como una obligación actual.

52. Riesgos principales

Riesgo 1 — Exceso de alcance

Mitigación:

MVP estricto.

Riesgo 2 — Nuevo `main_scene.gd` gigante

Mitigación:

responsabilidades pequeñas y paquetes independientes.

Riesgo 3 — Acoplar modelo a Three.js

Mitigación:

`model` sin dependencia visual.

Riesgo 4 — IFC como modelo operativo

Mitigación:

adaptador separado.

Riesgo 5 — OpenCascade demasiado pronto

Mitigación:

ADR obligatorio.

Riesgo 6 — Dependencia excesiva de agentes

Mitigación:

tareas pequeñas, autorización y validación humana.

Riesgo 7 — Infraestructura invisible durante demasiado tiempo

Mitigación:

cortes verticales y entregas visibles.

Riesgo 8 — Documentación que diverge del código

Mitigación:

todo comportamiento documentado como implementado deberá tener referencia verificable en código/test.

Riesgo 9 — Optimización prematura

Mitigación:

resolver primero comportamiento y UX.

53. Documentos que deberán existir antes del desarrollo real

README.md

docs/product/vision.md
docs/product/mvp-scope.md

docs/architecture/overview.md
docs/architecture/coordinate-system.md
docs/architecture/document-model.md
docs/architecture/geometry-policy.md

docs/migration/legacy-inventory.md
docs/migration/migration-rules.md

docs/validation/acceptance-matrix.md

docs/decisions/0001-web-first-local-first.md
docs/decisions/0002-parametric-document-source-of-truth.md
docs/decisions/0003-ifc-as-interchange-adapter.md
docs/decisions/0004-no-cad-kernel-in-mvp.md
docs/decisions/0005-visible-progress-and-vertical-slices.md
docs/decisions/0006-controlled-agent-changes.md

54. `.cursor/rules` iniciales

No se copiarán las reglas antiguas una por una.

La estructura inicial recomendada será:

```
.cursor/rules/  
├─ 00-project-architecture.mdc  
├─ 10-agent-behavior.mdc  
├─ 20-typescript.mdc  
├─ 21-react-ui.mdc  
├─ 22-threejs-viewer.mdc  
├─ 23-parametric-model.mdc  
├─ 24-geometry.mdc  
├─ 30-storage.mdc  
├─ 40-testing.mdc  
├─ 50-git.mdc  
├─ 60-documentation.mdc  
├─ 70-security-licenses.mdc  
└─ 80-legacy-migration.mdc
```

Cada regla deberá ser corta y normativa.

Los detalles extensos deberán vivir en documentación.

55. Qué NO debe hacer `.cursor`

No deberá contener:

- toda la arquitectura completa;
- todo el roadmap;
- toda la documentación de producto;
- explicaciones extensas;
- duplicación de ADR;
- código;
- decisiones experimentales.

Las reglas deberán decir **cómo debe comportarse el agente**.

Los documentos deberán explicar **por qué existe el sistema**.

56. Secuencia oficial de creación

No se comenzará directamente programando.

La secuencia será:

```
1. Auditoría aprobada
  ↓
2. Documento fundacional
  ↓
3. Decisiones abiertas resueltas
  ↓
4. Nuevo repositorio
  ↓
5. AGENTS.md
  ↓
6. .cursor/rules
  ↓
7. ADR
  ↓
8. Estructura mínima
  ↓
9. Etapa 0
  ↓
10. Primer muro
  ↓
11. MVP estricto
  ↓
12. Evaluación humana
```

57. Criterio de parada entre grandes etapas

Al terminar:

- Etapa 0;
- Etapa 1;
- MVP;

el proyecto deberá detenerse para evaluación.

No se continuará automáticamente a la siguiente gran funcionalidad.

Esto garantiza que el usuario pueda cambiar la dirección del producto antes de invertir semanas adicionales.

58. Principios no negociables

1. El documento paramétrico es la fuente de verdad.
 2. Three.js no es fuente de verdad del modelo.
 3. React no modifica directamente el documento.
 4. Toda mutación confirmada pasa por comandos.
 5. Las previsualizaciones no entran al historial.
 6. El trazado trabaja sobre un plano matemático.
 7. Cambiar de vista no modifica el modelo.
 8. IFC es un adaptador.
 9. OpenCascade no entra en el MVP.
 10. Cada fase produce una mejora visible.
 11. El usuario no necesita leer código para validar una entrega.
 12. No existen refactorizaciones masivas automáticas.
 13. La arquitectura no cambia sin autorización.
 14. Las tareas tienen alcance explícito.
 15. No se añaden funciones hipotéticas.
 16. No se modifican archivos no relacionados.
 17. Las pruebas no se eliminan para ocultar fallos.
 18. Una función de usuario no está terminada si no puede utilizarse desde la interfaz.
 19. El proyecto demo se mantiene actualizado.
 20. El agente se detiene al cumplir el objetivo.
-

59. Principio central de experiencia de desarrollo

El nuevo AxonBIM no debe construirse pensando únicamente:

"¿El código está bien?"

Debe preguntarse simultáneamente:

"¿El arquitecto puede verlo?"

"¿Puede probarlo?"

"¿Puede entenderlo?"

"¿Puede decidir si va en la dirección correcta?"

La calidad técnica es necesaria.

Pero para este proyecto:

La visibilidad del producto es también una condición de arquitectura.

60. Resultado final de la auditoría

La auditoría independiente confirma que el proyecto anterior contiene una base técnica reutilizable y que la mayoría de las conclusiones fundamentales obtenidas previamente están respaldadas.

También confirma que existen diferencias entre documentación e implementación, por lo que el nuevo proceso debe ser más estricto en la verificación de hechos.

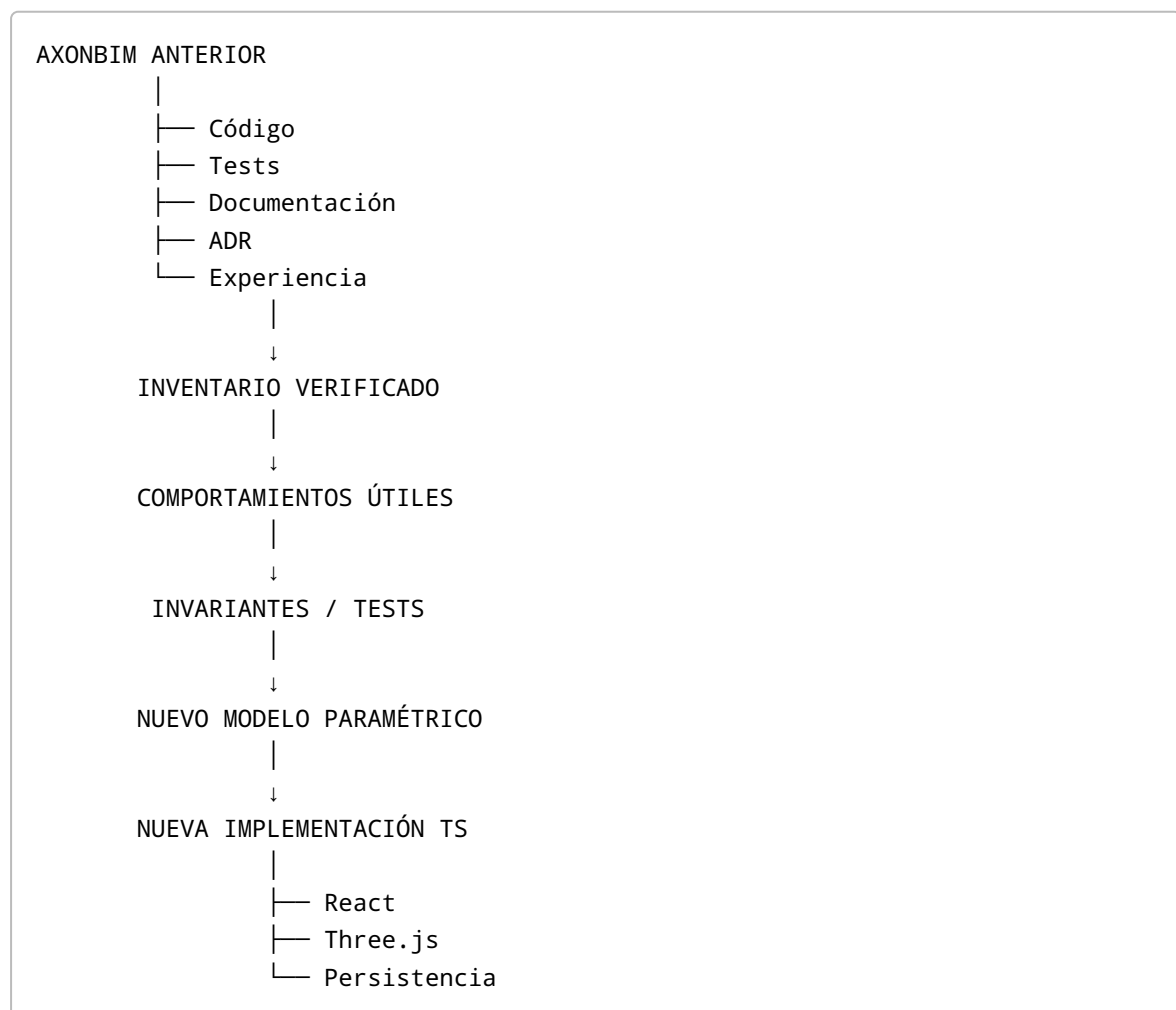
El proyecto anterior no debe considerarse un código que simplemente hay que traducir.

Debe considerarse:

una fuente de conocimiento, comportamiento, pruebas, errores y decisiones.

61. Decisión de migración definitiva

La estrategia aprobada es:



No habrá traducción automática del proyecto.

62. Primera meta real del nuevo proyecto

La primera meta no será:

"tener una arquitectura perfecta."

Ni:

"migrar todo AxonBIM."

Ni:

"implementar BIM."

Será:

crear una experiencia web convincente en la que un arquitecto pueda dibujar y modificar muros, verlos simultáneamente en planta y 3D, y comprobar que el nuevo modelo paramétrico funciona correctamente.

Si esa experiencia resulta natural y técnicamente sólida, se continúa.

Si no resulta natural, se corrige antes de añadir puertas, ventanas, losas, IFC o cualquier otra infraestructura.

63. Estado del plan

APROBADO COMO BASE CONCEPTUAL

- migración a web;
- TypeScript;
- modelo paramétrico;
- Three.js como representación;
- separación modelo/visualización;
- geometría analítica;
- IFC desacoplado;
- ausencia de OpenCascade en MVP;
- cortes verticales;
- MVP estricto;
- proyecto demo;
- validación visual;
- control de agentes.

DEBE DEFINIRSE ANTES DEL CÓDIGO

- tolerancias;
- sistema completo de coordenadas;
- contrato exacto de `AxonDocument`;
- política de IDs;
- contrato de comandos;
- formato JSON v1;
- estrategia exacta de serialización;
- política de migraciones;
- criterios geométricos de equivalencia;
- decisiones finales sobre React/Zustand;
- reglas iniciales de Cursor.

APLAZADO

- puertas;
- ventanas;
- losas;
- múltiples niveles;
- medición;
- IndexedDB;
- OPFS;
- PWA;
- IFC;
- DXF;
- documentación técnica;
- OpenCascade;
- colaboración;
- B-Rep general.

64. Conclusión

El objetivo de esta migración no es producir una versión web del código anterior.

El objetivo es producir una versión mejor fundamentada del producto.

El proyecto anterior ya respondió muchas preguntas difíciles:

- qué geometría funciona;
- cómo representar muros;
- cómo hacer snapping;
- cómo trabajar con herramientas;
- qué problemas genera IFC demasiado pronto;
- qué complejidad produce un puente RPC;
- qué ocurre cuando el frontend se centraliza;
- qué tipo de pruebas son útiles;
- qué problemas aparecen cuando el alcance crece demasiado.

AxonBIM Web utilizará esos aprendizajes para comenzar desde una base más pequeña.

La regla final será:

Primero una experiencia arquitectónica pequeña, visible y convincente. Después complejidad.

Y para los agentes:

Primero una tarea pequeña, autorizada y verificable. Después la siguiente.

El proyecto avanzará únicamente cuando el producto visible y la evidencia técnica indiquen que la etapa actual merece continuar.